Applied Machine Learning
COMP 551

A1 Report

Prof. Pascale Gourdeau
School of Computer Science

Steven Chang - 261239062
Mujtaba Mirhasan - 261213939
Zheng Ye Zhang - 261165863

McGill University, Montreal
4 February 2026

Abstract

This project investigates whether simple nonlinear feature transformations can improve the predictive performance of a linear regression model applied to the bike rental dataset. The original dataset was first cleaned by removing irrelevant columns for the target and verifying that no values were missing. Then, a simple linear regression model was implemented. Using an 80/20 random train–test split, controlled by a fixed seed for reproducibility, its performance was evaluated using the mean squared error (MSE). Doing this with many different seeds, the resulting mean MSE was 582009.35.

To explore the impact of feature engineering, several nonlinear transformations were added to the original feature set, including polynomial terms (e.g., temperature squared) and interaction terms (e.g., temperature $\times$ humidity). These engineered features allow a linear model to capture more complex relationships.

After retraining the model with the added features, the mean value of the MSE decreased to approximately 502949.17, indicating an improvement in predictive accuracy. This result suggests that even simple nonlinear transformations can enhance model performance without requiring more complex learning algorithms.

1 Introduction

The main goal of this project is to build and analyze a linear regression model to predict daily bike rental, based on meteorological conditions. After preparing the dataset, a linear regression model, implemented from scratch, is evaluated using the mean squared error (MSE). For this project, the regression weights are computed directly using the analytic (closed-form) solution to ordinary least squares linear regression, which is shown later in the report. The project then investigates whether simple non-linear feature transformations improve the model's ability to fit the data. This is done by retraining the model, collecting new results, and comparing the performance before and after feature engineering.

2 Data Preprocessing and Exploration

The dataset was first stored in a Pandas' DataFrame. Using Pandas' *.isna()* and *.any(axis=1)*, the rows were checked, one by one, for any NaN values. The DataFrame was found to have no NaN values. If there were some, the best option would have been to impute the NaN values with the mean. Imputing introduces more bias, since real variation in the data is replaced by a fixed value. However, keeping all rows helps the model to have a smaller variance, since removing rows makes the model more sensitive to the choice of data points used for the training. A smaller variance means that the model's predictions stay close to each other, even if the subset of rows chosen for the training is changed.

Then, the columns *instant* and *dteday* are removed because they do not add any predictive information about bike rental (*season*, *month*, *weekday*, and *year* already capture the information about the date). The columns *casual*, *registered*, and *cnt* need to be removed from the features, because they include the number of bike rentals, which is the target we want to predict.

Next, the numerical columns *yr*, *holiday*, and *workingday* were standardized so that they have zero mean and unit variance. Standardization ensures that these features contribute proportionally to the model and prevents variables with larger scales from dominating the regression. Additionally, the categorical columns *season*, *mnth*, *weekday*, and *weathersit* were transformed using one-hot encoding. This converts each categorical variable into a set of binary indicator variables, allowing the linear regression model to treat each category independently rather than assuming that there exists a numerical ordering between the values.

Finally, it is important to discuss feature correlations. From figure 2.1, when comparing the plot on the top-left and the one below it, there seems to be a linear correlation between the day's temperature and the number of rentals, ignoring the noise. Now, using the graph on the bottom-left, where *winter* = *1*, *spring* = *2*, *summer* = *3*, and *fall* = *4*, there appears to be a correlation between the season and the number of rentals. Of course, it makes sense that the number of rentals is lowest in the winter and at peaks during the summer. The weather may or may not affect the number of rentals; it is impossible to know without further analysis. The fluctuations in rentals with respect to weather may not be correlated at all. For daily humidity and wind speed, the values fall within approximately 0.4–0.9 and 0.1–0.26, respectively, with a few outliers. There was no apparent correlation at first glance, so more research is needed.

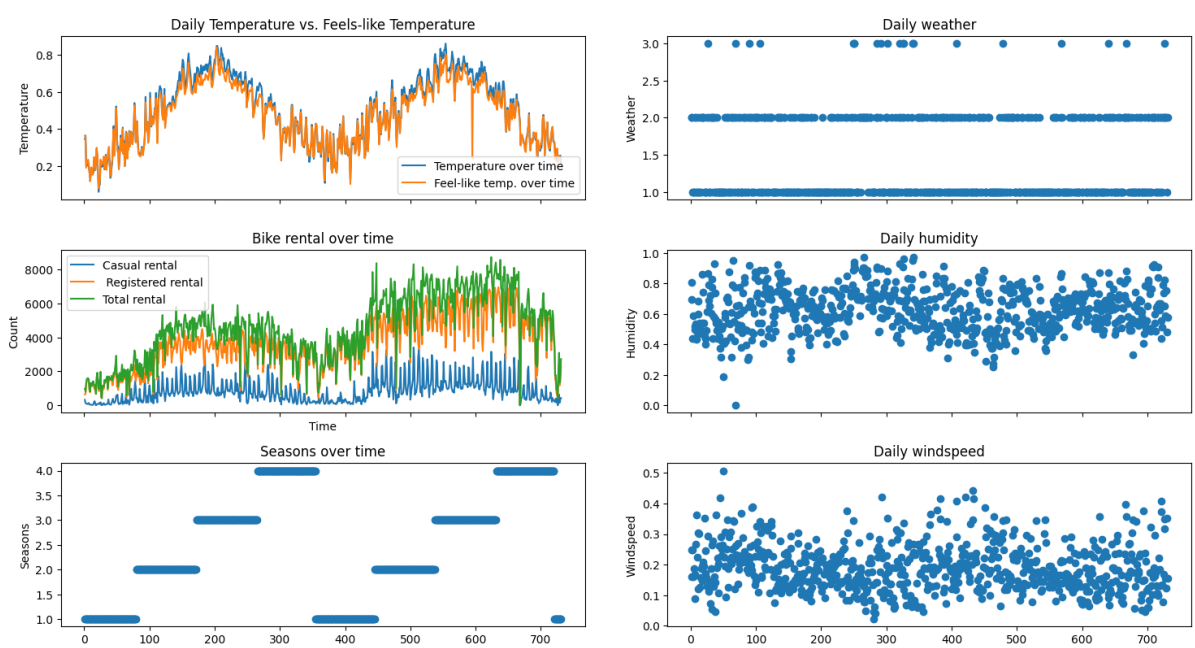

Figure 2.1: Plots for data visualization

3 Methods

Describe your linear regression implementation and any feature engineering approach you used.

The linear regression implementation is made by making a class called "LinearRegression". It contains two methods: fit(self, x_train, y_train) and predict(self, x_test).

Fit computes the optimal weight vector using the analytical least-squares solution. Instead of explicitly computing the inverse of $X^\top X$, the Moore–Penrose pseudoinverse is used via np.linalg.pinv. The weights are therefore obtained as $X^+\mathbf{y}$, where X^+ denotes the pseudoinverse of the design matrix. Using the pseudoinverse Using the pseudoinverse improves numerical stability and guarantees a solution even when $X^\top X$ is singular or ill-conditioned (see Appendix A for a proof).

The predict method then computes predictions for unseen data by applying the learned weights to the input features: $\hat{\mathbf{y}} = X_{test}\mathbf{w}$. To split the data into a training set and a test set, 80% of the dataset is allocated for training, while the remaining 20% is used for testing. The samples assigned to each set are selected randomly using a fixed seed with np.random.permutation, ensuring reproducibility.

3.1 Feature Engineering

In terms of feature engineering, we considered both creating new features by squaring existing ones and multiplying selected pairs of features. We mainly focused on input variables related to temperature, as our data preprocessing suggested that they have the highest correlating with our target variable.

In the case of polynomial features, we squared the normalized temperature (temp), the normalized feeling temperature (atemp) and the normalized humidity (hum) for each data sample and added these as new predictors. We multiplied the normalized temperature with the normalized humidity to create a new predictor. We also multiplied the normalized temperature with the binary feature that indicates whether the day is a working day (working day) to create a second interaction feature.

4 Results

To assess model performance, the MSE was computed across five different random seeds for both the original and engineered feature sets. This repeated evaluation helps ensure that the observed improvements are consistent and not dependent on a single train–test split. Across the five runs, the original feature set produced an average MSE of 582009.35, whereas the engineered feature set achieved a lower average MSE of 502949.17. These results,

summarized in Table 4.1, demonstrate that the engineered features consistently improve predictive accuracy relative to the baseline model.

Seed	**MSE (Original)**	**MSE (Engineered)**
15	607142.95	542972.99
10	655582.58	568648.93
9	549147.00	451907.21
8	606835.95	522751.05
7	491338.26	428465.69

Table 4.1: Comparison of test MSE across different random seeds.

5 Discussion and Conclusion

The results clearly show that introducing simple nonlinear transformations improves the performance of the linear regression model. Across the five seeds, the MSE with the engineered features consistently produced lower MSE values than the original features. This shows that the improvement is not a random outcome. With only a single MSE calculation, the engineered model might have appeared better simply due to a lucky split of the data.

Despite the fact that the model improve when new engineered features were added, it may still not be the most suitable approach: the squared terms and interactions terms expands the feature space, increasing the risk of overfitting, unstable coefficients and multicollinearity. These are issues that OLS struggles with, meaning that its performance or stability gets worse when these problems are present. However, there exists another model, the ridge regression, that is more robust than OLS when engineered features are used. This is because the L2 regularization (a technique used to keep the model's weights small so the model doesn't overfit) term penalizes large coefficients, stabilizes the model, and reduces variance. As a result, ridge regression can provide better generalization performance, especially in settings where feature engineering increases model complexity.

Statement of Contributions

Steven Chang coded Task 1. He wrote abstract, 1., 2., and 3. in the report. He also did Appendix A.
Mujtaba Mirhasan coded Task 2.
Zheng Ye interpreted the results and coded Task 3.

Statement on the use of LLMs and other resources

LLM was used to understand the several functions in Numpy, Pandas, and Matplolib. It also helped with the formatting of the document.

The team collaboration worked as follows: Steven completed Task 1 and sent the code to Mujtaba, who worked on Task 2 and then passed it to the final member for Task 3 (Zheng Ye). Everyone code reviewed each task and they could add to the document at any time.

A Using Moore-Penrose Pseudoinverse for Regression

The solution of Ordinary Least Squares (OLS) is $(X^\top X)^{-1}X^\top \mathbf{y}$, but this works if and only if $X^\top X$ is invertible. However, the pseudoinverse X^+ can be computed for any X. Thus, it would be necessary to prove that the pseudoinverse generalizes the OLS solution. In other words, show that $\mathbf{w} = (X^\top X)^{-1}X^\top \mathbf{y} = X^+\mathbf{y}$ when $X^\top X$ is invertible.

The optimization problem is the following

$$\min_w \|Xw - y\|^2$$

Then, do a Singular Value Decomposition (SVD) on X:

$$\min_w \|U\Sigma V^\top w - y\|^2$$

This makes the problem simpler because it is solved in a coordinate system where X is diagonal, meaning that there is no mixing between coordinates, so it decouples into independent 1D problems. Since U is orthogonal,

$$\begin{aligned}\|U\Sigma V^\top w - y\| &= \|U^\top(U\Sigma V^\top w - y)\| \\ &= \|\Sigma V^\top w - U^\top y\|\end{aligned}$$

Let $z = V^\top w$ and $b = U^\top y$. The problem becomes:

$$\min_z \|\Sigma z - b\|^2$$

Since Σ is diagonal,

$$\min_{z_i} (\sigma_i z_i - b_i)^2$$

Take the function:

$$f(z_i) = (\sigma_i z_i - b_i)^2$$

$$f'(z_i) = 2\sigma_i(\sigma_i z_i - b_i)$$

$$2\sigma_i(\sigma_i z_i - b_i) = 0 \Rightarrow \ \sigma_i z_i - b_i = 0$$

$$z_i = \begin{cases} \dfrac{b_i}{\sigma_i}, & \sigma_i \neq 0 \\ 0, & \sigma_i = 0 \end{cases}$$

Here, define

$$\Sigma^+_{ii} = \begin{cases} \dfrac{1}{\sigma_i}, & \sigma_i \neq 0 \\ 0, & \sigma_i = 0 \end{cases}$$

If $\sigma_i = 0$, then the derivative $2\sigma_i(\sigma_i z_i - b_i)$ becomes 0 regardless of z_i. This means that the error does not depend on z_i in that direction, so z_i can be any value. There is no way to

reduce the error in that direction. By convention, the value of $z_i = 0$.

Now, to go back to the original variables, recall that

$$z = V^\top w$$

$$\Rightarrow w = Vz$$

Notice that

$$\begin{aligned} w_i &= V_i \frac{1}{\sigma_i} b_i \\ \Rightarrow w &= V\Sigma^+ b \\ \Rightarrow w &= V\Sigma^+ U^\top y \end{aligned}$$

$$X^+ := V\Sigma^+ U^\top$$

This means that the general solution to an OLS is $w^* = X^+ y$. Here is the proof that this solution generalizes $w = (X^\top X)^{-1} X^\top y = X^+ y$:

$$\begin{aligned} X = U\Sigma V^\top &\Rightarrow X^\top X = V\Sigma U^\top U \Sigma V^\top = V\Sigma^2 V^\top \\ &\Rightarrow (X^\top X)^{-1} = V\Sigma^{-2} V^\top \\ &\Rightarrow (X^\top X)^{-1} X^\top = V\Sigma^{-2} V^\top V \Sigma U^\top = V\Sigma^{-1} U^\top \end{aligned}$$

Since X is square and full rank, $\Sigma^{-1} = \Sigma^+$ [1]:

$$(X^\top X)^{-1} X^\top = V\Sigma^{-1} U^\top = V\Sigma^+ U^\top = X^+$$

$$w = X^+ y = (X^\top X)^{-1} X^\top y$$

References

[1] K. Gilbert. *Least Squares*. pp. 13–14. University of California, Santa Barbara. n.d. URL: https://sites.cs.ucsb.edu/~gilbert/cs111/chapters/05_leastsquares.pdf.